

.data

Treatment

If a class contains public fields, use Encapsulate Field to hide them from direct access and require that access be performed via getters and .setters only

Use Encapsulate Collection for data stored in collections (such as .arrays)

Review the client code that uses the class. In it, you may find functionality that would be better located in the data class itself. If this is the case, use Move Method and Extract Method to migrate this .functionality to the data class

After the class has been filled with well thought-out methods, you may want to get rid of old methods for data access that give overly broad access to the class data. For this, Remove Setting Method and Hide .Method may be helpful

Payoff

Improves understanding and organization of code. Operations on particular data are now gathered in a single place, instead of .haphazardly throughout the code

.Helps you to spot duplication of client code

Data Class (کلاس داده)

علائم و نشانه‌ها (Signs and Symptoms)

کلاسی که فقط فیلدها و متدهای خام دسترسی (getter/setter) دارد. صرفاً ظرفی برای داده‌هاست که کلاس‌های دیگر استفاده می‌کنند. این کلاس‌ها هیچ عملکرد اضافی ندارند و نمی‌توان روی داده‌هاشون عملیات مستقل انجام بدن.

دلایل ایجاد (Reasons for the Problem) 🤔

دلیل	توضیح
کلاس جدید	طبیعیه که یه کلاس تازه ساخته شده فقط چند فیلد عمومی داشته باشه
قدرت واقعی اشیاء	شء باید رفتار و عملیات روی داده‌هاش رو هم داشته باشه

مثال 📝

csharp

```
// ❌ Data Class - فقط فیلد و getter/setter
public class Customer
{
    public string Name { get; set; }
    public string Email { get; set; }
    public decimal Balance { get; set; }
}

// منطقش wo کلاس‌های دیگه پخش شده
public class CustomerService
{
    public bool CanAfford(Customer c, decimal amount)
    {
        // این منطق باید توی Customer باشه!
        return c.Balance >= amount;
    }
}
```

درمان (Treatment) 🛠️

#	تکنیک	کاربرد
۱	Encapsulate Field	فیلدهای public رو خصوصی کن و getter/setter بذار

کاربرد	تکنیک	#
برای داده‌هایی که توی Collection هستن	Encapsulate Collection	۲
منطق پراکنده توی کد مشتری رو به خود Data Class منتقل کن	Move Method + Extract Method	۳
بعد از اضافه کردن متدهای منطقی، متدهای دسترسی قدیمی رو حذف یا مخفی کن	Remove Setting Method + Hide Method	۴

Payoff (دستاورد)

- بهبود فهم و سازماندهی کد. عملیات روی داده‌ها به جای پخش شدن، یه جا جمع میشن.
- تشخیص کد تکراری توی کلاینت‌ها.

بعد از درمان

csharp

```
public class Customer
{
    public string Name { get; set; }
    public string Email { get; set; }
    public decimal Balance { get; private set; } // Encapsulate Field

    // منطق منتقل شده به خود کلاس
    public bool CanAfford(decimal amount)
    {
        ;return Balance >= amount
    }

    public void Deposit(decimal amount)
    {
        ;Balance += amount
    }
}
```